

STUDY NOTES

Hugging Face LLM Course

Chapter 3 - Fine-Tuning a Pretrained Model

A structured reference covering dataset loading & preprocessing, dynamic padding, the high-level Trainer API, writing a full custom PyTorch training loop, distributed training with Hugging Face Accelerate, and reading learning curves - with a cheat sheet, glossary, interview questions, quiz, and hands-on exercises.

Hugging Face LLM Course - Chapter 3

Fine-Tuning a Pretrained Model - Study Notes

Concise reference notes covering loading and preprocessing datasets from the Hub, dynamic padding with data collators, fine-tuning via the Trainer API, writing a training loop from scratch, scaling with Hugging Face Accelerate, and diagnosing training runs via learning curves.

Table of Contents

1. Chapter Overview - Three Steps to Fine-Tuning
2. Processing the Data - Datasets, Tokenization, Dynamic Padding
3. Fine-Tuning with the Trainer API
4. A Full Training Loop - Custom PyTorch + Accelerate
5. Understanding Learning Curves
6. Chapter Summary
7. Cheat Sheet
8. Terminology
9. Acronyms
10. Interview Questions (20)
11. Mini Quiz - 20 MCQs
12. Hands-On Exercises
13. Revision Checklist

1. Chapter Overview - Three Steps to Fine-Tuning

OVERVIEW

Chapter 2 covered running inference on a pretrained model. Chapter 3 covers adapting one to a specific task: loading a real dataset from the Hub, preprocessing it efficiently, and training - first via the high-level **Trainer** API, then by hand with a custom PyTorch loop.

WHY IT MATTERS

Fine-tuning is the day-to-day work of most applied NLP/LLM engineers - pretraining a model from scratch is rare and expensive, but adapting an existing checkpoint to a company's data or task is common, cheap, and exactly what this chapter teaches end to end.

CORE CONCEPTS

- Three-step recipe, repeated throughout the chapter: **1)** load a dataset from the Hub, **2)** preprocess it with a tokenizer (plus dynamic padding), **3)** fine-tune with either Trainer or a manual training loop.
- The chapter is PyTorch-only - by this point in the course, PyTorch is treated as the standard framework for modern deep learning research and production.
- Everything is demonstrated on **MRPC** (Microsoft Research Paraphrase Corpus), a small sentence-pair dataset from the GLUE benchmark, chosen specifically because it's small enough to experiment with quickly.
- Two parallel paths are taught for the same result: the **Trainer API** (fast, few lines, handles logging/checkpointing/evaluation for you) and a **full custom loop** (complete control, needed for non-standard training logic or deep understanding).
- Hugging Face **Accelerate** is introduced as the bridge that takes a custom PyTorch loop and makes it run on multiple GPUs/TPUs with minimal code changes.

KEY TAKEAWAY

Fine-tuning always reduces to the same three moves - get data, preprocess it into model-ready tensors, and train - no matter whether you reach for Trainer's convenience or a hand-written loop's control.

EXAMPLE

Fine-tuning bert-base-uncased on MRPC to classify whether two sentences are paraphrases is the chapter's running example, used consistently across the Trainer-API and custom-loop sections so results can be directly compared.

DEVELOPER TIP - Start with the Trainer API for any standard classification/regression fine-tuning job - only drop to a custom loop when you need training logic Trainer genuinely can't express.

2. Processing the Data - Datasets, Tokenization, Dynamic Padding

OVERVIEW

This section takes raw text pairs all the way to padded, batch-ready tensors: loading a dataset from the Hub with the Datasets library, tokenizing sentence pairs correctly, mapping the tokenizer over the whole dataset efficiently, and padding dynamically per-batch instead of per-dataset.

WHY IT MATTERS

Inefficient preprocessing is one of the most common real-world training bottlenecks - padding every example to the dataset's longest sequence (instead of each batch's longest) can waste enormous amounts of compute on wasted padding tokens.

CORE CONCEPTS

- `load_dataset("glue", "mrpc")` returns a **DatasetDict** with train/validation/test splits, each holding sentence1, sentence2, label, and idx columns - indexable like a list of dicts.
- A dataset's `.features` attribute reveals column types; a **ClassLabel** feature stores the integer-to-string label mapping (e.g. 0 -> not_equivalent, 1 -> equivalent) directly.
- Tokenizing a **sentence pair** - `tokenizer(sentence1, sentence2)` - produces a single combined sequence shaped `[CLS] sentence1 [SEP] sentence2 [SEP]`, with `token_type_ids` marking which half each token belongs to (0 for the first sentence, 1 for the second).
- `token_type_ids` only appears for checkpoints pretrained with a next-sentence-prediction-style objective (e.g. BERT) - it's absent for others (e.g. DistilBERT); always let the matching tokenizer decide, don't hand-build it.
- `Dataset.map(tokenize_function, batched=True)` applies tokenization across the whole dataset efficiently - `batched=True` feeds many examples to the fast Rust tokenizer at once instead of one at a time, and the dataset stays disk-backed (Apache Arrow) rather than loaded fully into RAM.
- **Dynamic padding**: padding is deliberately left out of the mapped tokenize function and instead applied per-batch by a **collate function** - `DataCollatorWithPadding(tokenizer=tokenizer)` - so each batch only pads to its own longest sequence, not the dataset's longest.

KEY TAKEAWAY

Padding is a batching-time concern, not a dataset-preprocessing-time concern - tokenize without padding via `map(batched=True)`, then let a `DataCollatorWithPadding` pad each batch to just its own longest sequence for maximum efficiency.

EXAMPLE

A batch of 8 MRPC examples with raw lengths `[50, 59, 47, 67, 59, 50, 62, 32]` gets dynamically padded to length 67 (the batch's own max) - not to whatever the single longest sequence in the entire training set happens to be.

```
from datasets import load_dataset
from transformers import AutoTokenizer, DataCollatorWithPadding

raw_datasets = load_dataset("glue", "mrpc")
checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

```
def tokenize_function(example):  
    return tokenizer(example["sentence1"], example["sentence2"], truncation=True)  
  
tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)  
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

DEVELOPER TIP - Pass num_proc to Dataset.map() to parallelize preprocessing across CPU cores when you're not already using a Rust-backed fast tokenizer (which multithreads internally).

COMMON MISTAKE - Padding every example to the dataset's global max length inside the tokenize function instead of leaving padding to a per-batch data collator - this wastes compute on unnecessary padding tokens across the whole training run.

3. Fine-Tuning with the Trainer API

OVERVIEW

Transformers' Trainer class wraps the entire training loop - forward pass, loss, backward pass, optimizer step, evaluation, checkpointing, and logging - behind a configurable object, so fine-tuning a pretrained model on a labeled dataset takes only a few lines.

WHY IT MATTERS

Trainer encodes a large set of modern training best practices (mixed precision, sensible defaults, checkpoint/eval scheduling) so you don't have to reimplement and debug them yourself for standard fine-tuning jobs.

CORE CONCEPTS

- **TrainingArguments** is the configuration object: output directory, evaluation strategy (`eval_strategy="epoch"`), batch sizes, learning rate, number of epochs, logging, and more all live here.
- The model is loaded with a task head sized to the problem: `AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)` for MRPC's binary paraphrase task.
- `Trainer(model, training_args, train_dataset=..., eval_dataset=..., data_collator=..., processing_class=tokenizer)` wires everything together. Passing the tokenizer as `processing_class` auto-selects `DataCollatorWithPadding` as the default collator.
- `trainer.train()` runs the full loop; without an evaluation strategy set, no validation metrics are reported during training - only the training loss.
- `compute_metrics` is a user-supplied function that turns raw predictions + labels into named metrics (e.g. accuracy, F1 via the Evaluate library) - passing it (and setting `eval_strategy="epoch"`) makes Trainer report validation metrics at the end of every epoch alongside the training loss.
- Re-running training on the same model object without reinstantiating it just continues training further - always create a fresh model (and usually fresh `TrainingArguments`) for a clean new run.

KEY TAKEAWAY

Trainer is the fast path: define `TrainingArguments`, a model, your datasets, a data collator, and optionally `compute_metrics` - then `trainer.train()` does forward/backward/step/log/eval for every batch and epoch automatically.

EXAMPLE

Fine-tuning `bert-base-uncased` on MRPC for 3 epochs with Trainer and `compute_metrics` reports validation accuracy and F1 at the end of each epoch, letting you watch performance improve without writing any evaluation code yourself.

```
from transformers import TrainingArguments, Trainer, AutoModelForSequenceClassification

training_args = TrainingArguments("test-trainer", eval_strategy="epoch")
model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)

trainer = Trainer(
    model,
    training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
```

```
data_collator=data_collator,  
processing_class=tokenizer,  
compute_metrics=compute_metrics,  
)  
trainer.train()
```

DEVELOPER TIP - Small accuracy/F1 differences between runs on the same setup are expected - the classification head's weights are randomly initialized each time a fresh AutoModelForSequenceClassification is loaded.

COMMON MISTAKE - Calling `trainer.train()` again on the same Trainer/model expecting a fresh run - it resumes training the already-trained weights instead of starting over.

4. A Full Training Loop - Custom PyTorch + Accelerate

OVERVIEW

This section rebuilds the exact same fine-tuning result as the Trainer API, but by hand: a raw PyTorch DataLoader, model, optimizer, learning-rate scheduler, and an explicit forward-backward-step-zero_grad loop - then scales it to multiple GPUs/TPUs with Accelerate.

WHY IT MATTERS

Full manual control is needed whenever training logic goes beyond what Trainer exposes (custom losses, unusual data flows, research-y training tricks) - and understanding the loop underneath Trainer makes debugging any training run far easier.

CORE CONCEPTS

- **DataLoaders:** built from the tokenized dataset with `DataLoader(..., collate_fn=data_collator)`; shuffling is enabled for training only, and checking a batch's tensor shapes (e.g. `input_ids`, `attention_mask`, `labels`, `token_type_ids`) is a quick sanity check before training starts.
- **Optimizer:** AdamW (Adam with decoupled weight decay) is the standard choice for fine-tuning Transformer models.
- **Learning-rate scheduler:** a linear schedule that decays the learning rate over `num_training_steps = num_epochs * len(train_dataloader)` is standard practice alongside AdamW.
- **The core loop** per batch: move the batch to the target device -> forward pass -> read `outputs.loss` -> `loss.backward()` -> `optimizer.step()` -> `lr_scheduler.step()` -> `optimizer.zero_grad()`. Order matters - skipping `zero_grad()` silently accumulates gradients across steps.
- **Evaluation mode:** call `model.eval()` and wrap the forward pass in `torch.no_grad()` during evaluation - this disables dropout/batch-norm training behavior and gradient tracking, which is both more correct and much faster.
- **Accelerate** turns this single-device loop into a multi-GPU/TPU-ready one with minimal changes: wrap the model, optimizer, and dataloaders with `accelerator.prepare(...)`, swap `loss.backward()` for `accelerator.backward(loss)`, and drop the manual `.to(device)` calls - Accelerate handles device placement itself.

KEY TAKEAWAY

A training loop is always the same five moves in the same order - forward, backward, optimizer step, scheduler step, zero gradients - and Accelerate lets that exact loop scale from one GPU to many with only a few lines changed, not a rewrite.

EXAMPLE

The same MRPC fine-tuning job that took roughly 5 lines with Trainer takes roughly 20-30 lines by hand - but those lines are exactly what Trainer was doing internally, now fully visible and modifiable.

```
from torch.optim import AdamW
from transformers import get_scheduler
import torch

optimizer = AdamW(model.parameters(), lr=5e-5)
num_training_steps = num_epochs * len(train_dataloader)
lr_scheduler = get_scheduler("linear", optimizer=optimizer,
                             num_warmup_steps=0, num_training_steps=num_training_steps)
```



```
model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
```

DEVELOPER TIP - When something in a custom loop looks wrong, check the five-step order first (forward, backward, optimizer.step, scheduler.step, zero_grad) - an out-of-order or missing step is the most common source of silently broken custom training loops.

COMMON MISTAKE - Forgetting `optimizer.zero_grad()` at the end of the loop - gradients then accumulate across batches instead of being computed fresh each step, corrupting training.

5. Understanding Learning Curves

OVERVIEW

Once a model is training, watching loss and metric curves over time - rather than only the final number - is how you diagnose whether training is actually working, and if not, why.

WHY IT MATTERS

A single final accuracy number can't tell you whether a model is overfitting, underfitting, or simply needs more/fewer epochs - the shape of the curve over time carries that information.

CORE CONCEPTS

- Watch, during training: whether **loss is still decreasing** or has plateaued, whether **validation loss starts rising** while training loss keeps falling (overfitting), whether the learning-rate curve looks erratic (too high) or too flat (too low), and whether there are sudden spikes or drops.
- **Overfitting** signature: training performance keeps improving while validation performance stalls or gets worse - the model is memorizing training data rather than generalizing.
- **Underfitting** signature: both training and validation performance stay poor - often because the model is too small/simple, undertrained, or the learning rate is badly tuned.
- **EarlyStoppingCallback** (used with an eval strategy, `load_best_model_at_end=True`, and a chosen `metric_for_best_model`) automatically halts training once validation performance stops improving for a set number of evaluations - a direct, practical guard against overfitting.
- Experiment trackers (e.g. Weights & Biases via `report_to="wandb"`) turn logged metrics into interactive plots, making it far easier to spot these patterns than scrolling through printed step logs.

KEY TAKEAWAY

A learning curve is a diagnostic tool, not just a training log - reading the gap and trend between training and validation curves tells you whether to train longer, stop early, or rethink the model/data, before you ever look at a single final metric.

EXAMPLE

If training loss keeps dropping past epoch 3 while validation loss starts climbing, that's a clear overfitting signal - early stopping around the point validation loss bottoms out preserves the best-generalizing checkpoint instead of the most-memorized one.

```
from transformers import EarlyStoppingCallback

training_args = TrainingArguments(
    output_dir="./results", eval_strategy="steps", eval_steps=100,
    save_strategy="steps", save_steps=100, load_best_model_at_end=True,
    metric_for_best_model="eval_loss", greater_is_better=False,
    num_train_epochs=10,
)
trainer = Trainer(
    model=model, args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator, processing_class=tokenizer,
    compute_metrics=compute_metrics,
```

```
callbacks=[EarlyStoppingCallback(early_stopping_patience=3)],  
)
```

DEVELOPER TIP - Set eval_strategy to "steps" with a small eval_steps value on short/fast runs so you get enough points on the curve to actually see a trend, not just 2-3 epoch-level dots.

6. Chapter Summary

The whole chapter in five lines

- Fine-tuning is always: load a dataset from the Hub, preprocess it into model-ready tensors, then train on it.
- Tokenizing sentence pairs produces one combined [CLS] s1 [SEP] s2 [SEP] sequence with token_type_ids marking which half is which.
- Dynamic padding (tokenize without padding, then pad per-batch via DataCollatorWithPadding) is far more efficient than padding every example to the dataset's global max length.
- Trainer wraps the entire loop (forward/backward/step/log/eval/checkpoint) behind TrainingArguments and a few objects; a custom loop makes every one of those steps explicit and fully controllable.
- Accelerate scales a custom loop to multiple GPUs/TPUs with minimal code changes, and learning curves - not just final metrics - are how you diagnose overfitting, underfitting, and when to stop.

What clicks

- Padding is a batching-time decision, not a preprocessing-time one - that's the entire point of dynamic padding and DataCollatorWithPadding.
- Trainer isn't a different way to train - it's the same five-step loop (forward, backward, optimizer step, scheduler step, zero gradients) with the bookkeeping done for you.
- A learning curve's shape (gap between train/val, plateaus, spikes) carries information a single final metric can't.

Worth digging into further

- Sharing fine-tuned models and tokenizers back to the Hub - covered in Chapter 4.
- The Datasets library's full feature set (streaming, filtering, custom loading scripts) - Chapter 5.
- Parameter-efficient fine-tuning (LoRA/QLoRA/PEFT) for adapting very large models cheaply - referenced but not detailed here.

7. Cheat Sheet

NEED TO...	REACH FOR...
Load a Hub dataset	<code>load_dataset("glue", "mrpc")</code>
Check label meanings	<code>raw_train_dataset.features</code>
Tokenize a sentence pair	<code>tokenizer(sentence1, sentence2) -> input_ids + token_type_ids + attention_mask</code>
Tokenize a whole dataset efficiently	<code>dataset.map(tokenize_function, batched=True)</code>
Pad only to each batch's own max length	<code>DataCollatorWithPadding(tokenizer=tokenizer)</code>
Fine-tune with minimal code	<code>TrainingArguments + Trainer(...).train()</code>
Report validation metrics each epoch	<code>eval_strategy="epoch" + compute_metrics=...</code>
Build a raw PyTorch training loop	<code>DataLoader + AdamW + get_scheduler("linear", ...)</code>
Run the loop correctly per batch	<code>forward -> loss.backward() -> optimizer.step() -> lr_scheduler.step() -> zero_grad()</code>
Evaluate correctly (no dropout, no grad)	<code>model.eval() + torch.no_grad()</code>
Scale a custom loop to multi-GPU/TPU	<code>Accelerate: accelerator.prepare(...) + accelerator.backward(loss)</code>
Stop training once validation stalls	<code>EarlyStoppingCallback + load_best_model_at_end=True</code>

Three-line mental model

```
raw text pairs --tokenizer + map(batched=True)--> tokenized dataset (unpadded)
tokenized dataset --DataLoader + DataCollatorWithPadding--> padded batches
padded batches --Trainer.train() OR manual loop + Accelerate--> fine-tuned model
```

Decision table

Question to ask	Answer tells you
Is this a standard classification/regression fine-tune?	Use the Trainer API
Do I need custom loss functions or non-standard training logic?	Write a custom loop
Do I need to run on multiple GPUs/TPUs?	Wrap the custom loop with Accelerate
Is validation loss rising while training loss falls?	Overfitting - consider early stopping
Are both train and validation metrics staying poor?	Underfitting - check model size, LR, epochs

8. Terminology

Term	Working definition
DatasetDict	A dict-like container holding a dataset's train/validation/test splits together.
ClassLabel	A dataset feature type storing the integer-to-string mapping for a categorical label column.
token_type_ids	IDs marking which of two sentences in a pair each token belongs to (0 or 1); only present for checkpoints pretr
Dataset.map()	Applies a preprocessing function across an entire dataset, optionally in fast batched mode.
Dynamic padding	Padding each batch only to its own longest sequence, applied at batch-assembly time rather than across the v
Collate function	The function that assembles individual examples into a single padded batch tensor.
DataCollatorWithPadding	A ready-made collate function that dynamically pads a batch using the correct tokenizer settings.
TrainingArguments	The configuration object for Trainer: output dir, eval/save strategy, batch size, learning rate, epochs, etc.
Trainer	A high-level class that runs the full fine-tuning loop (train, evaluate, log, checkpoint) given a model, data, and c
compute_metrics	A user-defined function converting raw predictions and labels into named evaluation metrics.
AdamW	Adam optimizer with decoupled weight decay; the standard optimizer for fine-tuning Transformers.
Learning-rate scheduler	A schedule that adjusts the learning rate over training steps, commonly linear decay for fine-tuning.
Accelerate	A Hugging Face library that adapts a PyTorch training loop to run on multiple GPUs/TPUs with minimal code c
Overfitting	When a model's training performance keeps improving while its validation performance stalls or worsens.
Underfitting	When a model fails to learn the underlying patterns, showing poor performance on both training and validation
Early stopping	Halting training automatically once a monitored validation metric stops improving for a set patience window.
Learning curve	A plot of a metric (loss, accuracy, etc.) over training time/steps, used to diagnose training behavior.

9. Acronyms

Acronym	Stands for
GLUE	General Language Understanding Evaluation (benchmark)
MRPC	Microsoft Research Paraphrase Corpus
CLS	(Not strictly an acronym) - BERT's classification special token
SEP	(Not strictly an acronym) - BERT's separator special token
NSP	Next Sentence Prediction (a BERT pretraining objective)
AdamW	Adam optimizer With decoupled weight decay
LR	Learning Rate
TPU	Tensor Processing Unit
GPU	Graphics Processing Unit
F1	F1 score - harmonic mean of precision and recall
W&B	Weights & Biases (an experiment tracking tool)
PEFT	Parameter-Efficient Fine-Tuning

10. Interview Questions (20)

1. What are the three main steps in fine-tuning a model, per this chapter?

Load a dataset from the Hub, preprocess it with a tokenizer (including padding strategy), and train the model on it using either the Trainer API or a custom training loop.

2. Why is MRPC a good teaching dataset for this chapter?

It's small (a few thousand sentence pairs), which makes it fast and cheap to experiment with while still being a real labeled classification task from the GLUE benchmark.

3. What does tokenizing a sentence pair produce differently from tokenizing one sentence?

A single combined sequence in the form [CLS] sentence1 [SEP] sentence2 [SEP], plus token_type_ids marking which tokens belong to which sentence.

4. Why don't all checkpoints return token_type_ids?

Only checkpoints pretrained with an objective that uses sentence-pair information (like BERT's next sentence prediction) expect and return them; others, like DistilBERT, don't.

5. What does Dataset.map(fn, batched=True) do, and why use batched=True?

It applies a preprocessing function across the dataset; batched=True passes multiple examples at once to the function, which lets a fast Rust-backed tokenizer process them far quicker than one at a time.

6. What is dynamic padding, and why is it more efficient than padding everything upfront?

Padding each batch only to its own longest sequence at batch-assembly time, rather than padding every example to the dataset's global max length - this avoids wasting compute on unnecessary padding tokens.

7. What does DataCollatorWithPadding need to know, and why?

It needs the tokenizer, so it knows which token ID to use for padding and whether the model expects padding on the left or right side of the sequence.

8. What does TrainingArguments configure?

Training configuration such as the output directory, evaluation/save strategy, batch sizes, learning rate, number of epochs, and logging behavior.

9. What happens if you don't pass compute_metrics or set an eval strategy to Trainer?

Trainer will only report training loss during training; it won't compute or display validation metrics along the way.

10. Why might re-running trainer.train() on the same model object give confusing results?

It continues training the already fine-tuned weights instead of starting fresh, so results reflect additional training on top of the previous run, not a clean new run.

11. List the five ordered steps inside one manual training loop iteration.

Forward pass, compute loss, loss.backward(), optimizer.step(), lr_scheduler.step(), then optimizer.zero_grad() - done in that order, every batch.

12. What happens if you forget optimizer.zero_grad() in a custom loop?

Gradients accumulate across batches instead of being freshly computed each step, corrupting the training dynamics.

13. Why call model.eval() and wrap forward passes in torch.no_grad() during evaluation?

model.eval() disables training-only behaviors like dropout; torch.no_grad() disables gradient tracking, making evaluation both more correct and significantly faster.

14. What optimizer is standard for fine-tuning Transformer models, and why?

AdamW - Adam with decoupled weight decay - is the standard choice because it works well with Transformer architectures and large parameter counts.

15. What does Accelerate let you do that a plain PyTorch loop doesn't, easily?

Run the same training loop across multiple GPUs or TPUs with minimal code changes, without manually managing device placement and distributed communication yourself.

16. What specifically changes when converting a manual loop to use Accelerate?

Model, optimizer, and dataloaders get wrapped with `accelerator.prepare(...)`, `loss.backward()` becomes `accelerator.backward(loss)`, and manual `.to(device)` calls are removed since Accelerate handles device placement.

17. What's the key sign of overfitting in a learning curve?

Training loss/metric keeps improving while validation loss/metric plateaus or gets worse - the model is increasingly memorizing rather than generalizing.

18. What's the key sign of underfitting?

Both training and validation performance remain poor, often pointing to too small a model, too few training steps, or a badly tuned learning rate.

19. How does EarlyStoppingCallback help, mechanically?

It monitors a chosen validation metric and halts training automatically once that metric stops improving for a set number of evaluation rounds, working alongside `load_best_model_at_end` to keep the best checkpoint.

20. Why prefer Trainer for a standard fine-tuning job instead of writing a custom loop?

Trainer already encodes common best practices (evaluation scheduling, checkpointing, logging, mixed precision support) correctly, so you get a working, well-tested loop in a few lines instead of reimplementing and debugging it yourself.

11. Mini Quiz - 20 MCQs

1. The three-step fine-tuning recipe is:

- A. Train, test, deploy
- B. Load dataset, preprocess, train
- C. Tokenize, pretrain, quantize
- D. Download, compress, ship

Answer: B

2. `load_dataset("glue", "mrpc")` returns:

- A. A single tensor
- B. A DatasetDict with train/validation/test splits
- C. A trained model
- D. A tokenizer

Answer: B

3. Tokenizing a sentence pair produces which special-token pattern?

- A. [CLS] only
- B. [CLS] s1 [SEP] s2 [SEP]
- C. [SEP] s1 [CLS] s2
- D. No special tokens

Answer: B

4. `token_type_ids` indicate:

- A. Token position
- B. Which sentence a token belongs to
- C. Padding vs. real tokens
- D. Vocabulary frequency

Answer: B

5. `batched=True` in `Dataset.map()` mainly:

- A. Reduces accuracy
- B. Speeds up tokenization by processing many examples at once
- C. Automatically pads
- D. Converts to PyTorch tensors

Answer: B

6. Dynamic padding pads each batch to:

- A. The full dataset's max length
- B. A fixed length of 512 always
- C. That batch's own longest sequence
- D. Zero length

Answer: C

7. `DataCollatorWithPadding` needs which argument to work correctly?

- A. The raw dataset
- B. The tokenizer
- C. The model
- D. The optimizer

Answer: B

8. TrainingArguments configures:

- A. Model architecture only
- B. Output dir, eval strategy, batch size, LR, epochs, etc.
- C. Only the dataset
- D. Only the tokenizer

Answer: B

9. Passing a tokenizer as processing_class to Trainer:

- A. Disables padding
- B. Auto-selects DataCollatorWithPadding as the default collator
- C. Trains a new tokenizer
- D. Has no effect

Answer: B

10. Without eval_strategy set, Trainer reports:

- A. Nothing at all
- B. Only training loss, no validation metrics
- C. Only validation metrics
- D. Test set accuracy automatically

Answer: B

11. The correct order of steps in a manual training loop iteration is:

- A. zero_grad -> forward -> backward -> step
- B. forward -> backward -> optimizer.step -> scheduler.step -> zero_grad
- C. backward -> forward -> step -> zero_grad
- D. Order doesn't matter

Answer: B

12. Forgetting optimizer.zero_grad() causes:

- A. Faster training
- B. Gradients to accumulate incorrectly across batches
- C. No effect
- D. The model to reset

Answer: B

13. During evaluation you should use:

- A. model.train() and torch.no_grad()
- B. model.eval() and torch.no_grad()

- C. `model.eval()` only, with gradients on
- D. Neither eval nor `no_grad`

Answer: B

14. The standard optimizer for fine-tuning Transformers is:

- A. SGD
- B. AdamW
- C. RMSProp
- D. Adagrad

Answer: B

15. Accelerate is primarily used to:

- A. Speed up tokenization
- B. Scale a custom training loop across multiple GPUs/TPUs
- C. Replace the Trainer API entirely
- D. Compress model weights

Answer: B

16. Converting a loop to use Accelerate typically involves:

- A. Rewriting the model architecture
- B. Wrapping objects with `accelerator.prepare()` and using `accelerator.backward()`
- C. Removing the optimizer
- D. Switching to TensorFlow

Answer: B

17. Validation loss rising while training loss falls is a sign of:

- A. Underfitting
- B. Overfitting
- C. Perfect convergence
- D. A broken tokenizer

Answer: B

18. Both training and validation metrics staying poor throughout training suggests:

- A. Overfitting
- B. Underfitting
- C. The dataset is too large
- D. Early stopping should never be used

Answer: B

19. EarlyStoppingCallback works by:

- A. Stopping training at a fixed epoch always
- B. Halting once a monitored validation metric stops improving
- C. Increasing the learning rate
- D. Deleting the worst checkpoint

Answer: B

20. A key reason to prefer Trainer over a custom loop for standard fine-tuning is:

- A. It's the only way to use GPUs
- B. It already encodes common best practices correctly, saving reimplementation
- C. Custom loops can't use AdamW
- D. Trainer requires no dataset

Answer: B

12. Hands-On Exercises

Exercise 1 - Load and Inspect a New Dataset

Load the GLUE SST-2 dataset (single-sentence, not pairs). Inspect its features and print element 10 from the training set. Note how it differs structurally from MRPC.

Exercise 2 - Write a Tokenize Function for SST-2

Write a `tokenize_function` for SST-2 (single-sentence input, unlike MRPC's pairs) and apply it with `dataset.map(fn, batched=True)`. Confirm the resulting columns match what you expect.

Exercise 3 - Dynamic Padding by Hand

Grab 8 examples from a tokenized dataset, remove non-tensor columns, and manually inspect each example's `input_ids` length. Then run them through `DataCollatorWithPadding` and confirm every tensor shares the batch's max length.

Exercise 4 - Trainer vs. Custom Loop

Fine-tune the same checkpoint on the same dataset once with `Trainer` and once with a custom loop (same epochs, batch size, and learning rate). Compare final validation accuracy and note any differences and why they might occur.

Exercise 5 - Diagnose a Learning Curve

Given a plotted training/validation loss curve where validation loss bottoms out at epoch 4 then rises while training loss keeps falling through epoch 10, write two sentences describing what's happening and what configuration change (e.g. `EarlyStoppingCallback`) would fix it.

Exercise 6 - Accelerate Conversion Drill

Take a plain PyTorch training loop (`forward/backward/step/zero_grad`) and rewrite it using `Accelerate`, listing each line you changed and why.

13. Revision Checklist

- Name the three steps of the fine-tuning recipe used throughout this chapter.
- Explain what `token_type_ids` represent and when they appear.
- Explain why `Dataset.map(fn, batched=True)` speeds up preprocessing.
- Explain dynamic padding and why it's more efficient than padding the whole dataset upfront.
- List the objects passed into a `Trainer` instantiation.
- Explain what `compute_metrics` does and when it's called.
- Recite the five ordered steps of one manual training loop iteration.
- Explain why `model.eval()` and `torch.no_grad()` matter during evaluation.
- Explain what changes when converting a custom loop to use `Accelerate`.
- Distinguish overfitting from underfitting using learning-curve shape.
- Explain how `EarlyStoppingCallback` decides when to stop training.
- Name at least 10 terms from the glossary from memory.
- Score at least 16/20 on the mini quiz without looking back.
- Re-do the mini quiz cold after a couple of weeks - a lower score is a signal to revisit this notebook.

Reference: Hugging Face LLM Course - Chapter 3 (huggingface.co/learn/llm-course/chapter3)